EE6497 Quiz 2 复习资料（Weeks 8–11）

Pattern Recognition and Deep Learning • Wang Lipo 部分

> 适用：EE6497（PG）/ IE4497（UG）Quiz 2
>
> 覆盖：**Week 8 – Week 11**
>
> 整理日期：2026-11-01
>
> 来源：`6497/Learning_note.md`（Week 8/9/11）、`Part 2.1` / `Part 2.2` 课件、`quiz2/EE6497_Practice_Quiz_2.html` 与 `quiz2/Q2_practice_逐项解析.txt`。

---

0. Quiz 2 要求（先看规则）

项目	内容
Quiz 2 占比	**10%**；课程总评：PG = Quiz 1 10% + Quiz 2 10% + Final 60% + HW 10% + Participation 5% + Survey 5%；UG = Participation 10%，无 Survey Report
时间	**Week 12（11/2–11/6）课上**；Week 11 老师原话：下次课 **21:00 左右**开始；具体日期/教室以 NTULearn 为准
形式	**online via NTULearn**，但必须到 **lecture theater**，自带 laptop 完成
范围	**Week 8–11**（Neural Networks → Deep Learning/CNN → Training Deep Networks ×2）
允许材料	**一张双面 A4 reference sheet**（手写或打印均可；**不允许 sticky notes/post-it**）
不考	**Python 编程**（老师明确说 "not in the final exam or in the quiz"）
练习	老师已发 **Practice Quiz 2**：**13 道 MCQ**；正式 quiz 题量/难度类似
缺考	无有效 LOA/MC → **0 分**

> ⭐ **策略**：Quiz 2 是 MCQ 为主，考的是**概念边界 + 小计算 + 易混点辨析**。复习优先级：
> **① Practice Quiz 13 题全部弄懂 → ② W8–W11 的公式/定义表 → ③ A4 reference sheet 抄熟 → ④ 手算 forward/backprop。**

---

1. 10 分钟知识地图

周	主题	核心内容	Quiz 高频点
W8	Neural Networks 导论	neuron model、activation functions、perceptron、linear separability、XOR、MLP、universal approximation	2-layer 术语、sigmoid/tanh/ReLU/Leaky ReLU、XOR 为什么单层不行、UAT 的保证与不保证
W9	多层网络设计 + Forward Propagation	4+2+1 设计、XOR 最小解、decision region、forward pass、parameters vs hyperparameters	1/2/3 层能表示什么区域、forward 手算、UAT 条件、training vs inference
W10	Training Deep Networks（前半）	MLP forward、softmax、loss、gradient descent、backprop、mini-batch/SGD、momentum/Adam、learning rate、vanishing gradient	sigmoid 饱和、learning rate 过大/过小、softmax 性质、CE loss、backprop 公式
W11	Training（后半）+ CNN 导论	generalization、L1/L2、dropout、early stopping、batch norm、hyperparameter tuning、K-fold、CNN	overfitting 曲线、L1 vs L2、dropout 推理、batch norm、UAT、CNN vs MLP

2. Week 8 — 神经网络基础

2.1 单个 neuron 的数学

一个 artificial neuron：

```
z = w1*x1 + w2*x2 + ... + wn*xn + b = w^T x + b
a = f(z)
```

- `x`：输入向量；`w`：weights；`b`：bias；`f`：activation function；`a`：activation/output。
- **bias 可以看成额外输入** `x0 = 1`，对应权重 `w0 = b`。
- 多层网络：

```
a^(l) = f( W^(l) a^(l-1) + b^(l) )
```

- 所有层堆起来：`y = f_L( W^L ... f_2( W^2 f_1( W^1 x + b^1 ) + b^2 ) ... + b^L )`。

2.2 Activation functions（必背表）

函数	公式	值域	导数	特点 / 考点
Sigmoid	`σ(z) = 1 / (1 + e^(-z))`	(0,1)	`σ(z)(1-σ(z))`	输出恒正、非 zero-centered；两端**饱和**，导数→0 → **vanishing gradient**
Tanh	`tanh(z) = (e^z - e^(-z)) / (e^z + e^(-z)) = 2σ(2z) - 1`	(-1,1)	`1 - tanh^2(z)`	**zero-centered**，通常优于 sigmoid；但仍会饱和
ReLU	`max(0, z)`	[0, ∞)	`1 (z>0)`；`0 (z<0)`	计算快、缓解 vanishing gradient；负半轴梯度 0 → **dying ReLU**
Leaky ReLU	`z (z>0)`；`αz (z<0)`，α 小正数（如 0.01）	(-∞, ∞)	`1 (z>0)`；`α (z<0)`	负半轴有**小非零梯度** → 缓解 dying ReLU
Linear	`f(z) = c z`（c=1 为 identity）	ℝ	`c`	回归输出层；不引入非线性
Softmax	`p_i = e^(z_i) / Σ_j e^(z_j)`	(0,1)，Σ=1	—	多分类**输出层**；把 logits 变成概率分布；**不用于 hidden layer**

⭐ **必记辨析**：

- **Sigmoid vs Tanh**：tanh 是 **zero-centered**（输出 -1 到 1），sigmoid 输出全正；tanh 常让梯度更新更平衡。但 tanh **仍会饱和**，不能"完全消除" vanishing gradient。
- **ReLU vs Leaky ReLU**：ReLU 负半轴导数 0，可能"死亡"；Leaky ReLU 负半轴斜率 α，梯度不全为 0。
- **Softmax vs Sigmoid**：softmax 的分母耦合所有 logits，输出**和为 1**；sigmoid 每个输出独立，不保证和为 1。
- **Sigmoid 饱和**：z→±∞ 时 σ→0/1，导数→0；深层网络连乘小导数 → 早层梯度消失。

2.3 Perceptron 与 decision boundary

- 单层 perceptron：`y = sign(w^T x + b)`。
- Decision boundary：`w^T x + b = 0`，是一条**直线/超平面**。
- `w` 决定方向，`b` 决定偏移；bias 可以让边界**不经过原点**。
- Decision region：半空间（half-plane / half-space）。

2.4 Linear separability 与 XOR

- **Linear separable**：存在一条直线/超平面把两类完全分开。
- **XOR**：

x1	x2	y
0	0	0
0	1	1
1	0	1
1	1	0

- XOR 的两类点呈"对角"分布，**任何一条直线都无法分开** → 单层 perceptron 不可能解决 XOR。

- ⭐ **同一层多放几个 neuron 也救不了**：单层多个 neuron 仍然是输入的各线性组合，决策边界仍由 hyperplane 组成；没有 hidden layer + 非线性，就无法组合出 XOR 需要的非线性区域。

2.5 多层解法：XOR 与 4+2+1

XOR 的最小 2 层 3-neuron 解（step activation）：

```
hidden #1:  z1 = x1 + x2 - 0.5      → OR
hidden #2:  z2 = -x1 - x2 + 1.5     → NAND
output:     y  = h1 + h2 - 1.5      → AND
```

即 **XOR = (x1 OR x2) AND NOT(x1 AND x2)**。用 2 层（1 hidden + output）即可。

4+2+1 网络（两个矩形区域的分类）：

- 第一层 4 个 neuron：分别切出两个矩形的上/下/左/右边界（每个 neuron 是一个 half-plane）；
- 第二层 2 个 neuron：分别对两个矩形的 4 个条件做 **AND**；
- 第三层 1 个 neuron：对两个矩形做 **OR**。

⭐ 一般模式：**OR-of-ANDs / AND-of-ANDs**；3 层（2 hidden + output）可以表示任意（非凸）区域。

2.6 Universal Approximation Theorem（UAT）

定理（简化）：一个**单 hidden layer**、足够多 hidden neurons 的 feedforward network，在合适的 activation（非恒定、有界、连续等条件）下，可以以任意精度逼近**紧致域上的任意连续函数**。

层数	能表示的区域
1 层（无 hidden）	单个 **half-plane**（线性边界）
2 层（1 hidden + output）	单个 **convex region**（凸区域/凸多边形）
3 层（2 hidden + output）	**任意区域**（非凸、多个分离区域）

⭐ **UAT 保证什么 / 不保证什么**：

- ✅ 保证：**存在**一组参数，使网络能逼近目标函数到任意精度（只要 hidden neurons 足够多）。
- ❌ 不保证：gradient descent 一定能找到这组参数；不保证全局最优；不保证泛化；不保证训练稳定；不保证"单层一定比多层高效"。
- ⭐ 常见陷阱：把 UAT 说成"保证训练一定成功"或"保证不会 overfit"——都不对。

2.7 Forward propagation（手算）

对一层：

```
z = W x + b
a = f(z)
```

例（Week 9 自测）：

```
W1 = [[1, -2],
      [-1, 1]]
b1 = [1, 0]^T
x  = [1, -1]^T

z1 = W1 x + b1 = [1*1 + (-2)*(-1) + 1,  (-1)*1 + 1*(-1) + 0]^T
   = [4, -2]^T
a1 = sigmoid(z1) ≈ [0.982, 0.119]^T
```

⭐ 手算要点：

- 矩阵乘法按行做内积；
- bias 逐元素加；
- sigmoid 逐元素算；
- 注意维度：`W` 是 `(out_dim, in_dim)`，`x` 是 `(in_dim, 1)`。

2.8 术语与参数

术语	含义
2-layer neural network	**1 个 hidden layer + 1 个 output layer**；**input layer 不计入**（它不做可学习的仿射变换）
3-layer neural network	2 个 hidden + 1 个 output
Parameters	`W, b`，由训练（backprop）学习
Hyperparameters	层数、每层 neuron 数、learning rate、batch size、λ、dropout rate、activation 选择等，训练前设定
Training	forward + backward + 参数更新
Inference	冻结参数，只做 forward；dropout/batch norm 行为不同

2.9 Week 8 高频陷阱

陷阱	正确说法
"2-layer = 2 hidden layers"	❌ 常见约定是 **1 hidden + output**；input 不计
"tanh 不会饱和"	❌ tanh 仍饱和；只是 zero-centered
"ReLU 在正半轴饱和"	❌ ReLU 正半轴是线性的，导数为 1；零梯度在负半轴
"Leaky ReLU 增加 bias"	❌ 它改变负半轴斜率，不新增 bias
"sigmoid 输出可以为负"	❌ sigmoid ∈ (0,1)，始终为正
"UAT 保证训练收敛到全局最优"	❌ UAT 只保证存在性/表达能力

---

3. Week 9 — 多层网络、Forward Propagation、UAT

3.1 Decision region 与层数

网络	决策区域
单个 neuron / 单层	一个 **half-plane**（线性）
2 层（1 hidden）	一个 **convex region**（凸区域；多个 AND 可以形成凸多边形）
3 层（2 hidden）	**任意区域**（OR 组合多个凸区域 → 非凸、多个分离区域）

⭐ 记忆：**1 层切一刀，2 层围一块，3 层拼任意形状**。

3.2 4+2+1 网络的设计

目标：分类两个矩形区域（例如左上和右下）。

- **第一层 4 个 neuron**：
- neuron 1：`x1 >= a`（切左边界）
- neuron 2：`x1 <= b`（切右边界）
- neuron 3：`x2 >= c`（切下边界）
- neuron 4：`x2 <= d`（切上边界）
- 每个 neuron 是一个 half-plane；4 个 AND 起来就是一个矩形。
- **第二层 2 个 neuron**：
- neuron 5：对矩形 1 的 4 个条件做 **AND**；
- neuron 6：对矩形 2 的 4 个条件做 **AND**。
- **第三层 1 个 neuron**：
- neuron 7：对两个矩形做 **OR**。

⭐ 关键：**AND 可以用一个 neuron 实现**（所有条件同时满足才激活）；**OR 也可以用一个 neuron 实现**（任一条件满足就激活）。这就是"多层 = 多个线性边界 + 逻辑组合"。

3.3 Forward propagation 的维度

对第 `l` 层：

```
z^(l) = W^(l) a^(l-1) + b^(l)
a^(l) = f^(l)(z^(l))
```

- `W^(l)`：`(n_l, n_{l-1})`；
- `b^(l)`：`(n_l, 1)`；
- `a^(l-1)`：`(n_{l-1}, 1)`；
- `z^(l), a^(l)`：`(n_l, 1)`。

⭐ 手算顺序：`W x` → `+ b` → `activation`；不要漏 bias，不要弄反矩阵维度。

3.4 Parameters vs Hyperparameters

	Parameters	Hyperparameters
例子	`W, b`	层数、每层宽度、learning rate、batch size、λ、dropout rate、activation、optimizer
如何确定	训练（backprop）	训练前设定 / 用 validation 调
能否用梯度下降学	✅	❌（不能直接对"层数"求导）

3.5 Training vs Inference

	Training	Inference
参数	更新	冻结
Dropout	随机丢弃	**关闭**（`.eval()`）
Batch norm	用当前 mini-batch 统计量	用训练时累计的 running mean/var
目的	学参数	预测

3.6 Week 9 高频陷阱

陷阱	正确说法
"单层多放 neuron 就能解 XOR"	❌ 单层仍是线性边界；需要 hidden layer + 非线性
"2 层网络能表示任意区域"	❌ 2 层只能表示**凸区域**；任意区域需要 3 层
"UAT 保证单层比多层好"	❌ UAT 不比较效率；深层有时用更少参数
"UAT 保证能找到参数"	❌ 只保证存在性
"bias 不属于参数"	❌ bias 是 parameter，也要训练

---

4. Week 10 — Training Deep Networks（前半）

4.1 MLP forward 与 softmax

- 分类网络：最后一层输出 **logits** `z`，再用 **softmax** 变成概率：

```
p_i = e^(z_i) / Σ_j e^(z_j)
```

- 性质：`p_i > 0`，`Σ_i p_i = 1`；最大 logit 对应最大概率，但 softmax **不是** one-hot。
- 二分类可以用 **sigmoid** 输出一个概率；多分类用 **softmax**。

4.2 Loss functions

Cross-entropy（分类）：

```
L = -(1/N) Σ_i Σ_k y_ik log( p_ik )
```

其中 `y` 是 one-hot / ground truth，`p` 是 softmax 输出。

MSE（回归）：

```
L = (1/N) Σ_i ( y_i - y_hat_i )^2
```

⭐ PyTorch 的 `cross_entropy` 例子（课件）：输入 `y = [0.8, 0.1, 0.06, 0.04]` 被当作 **logits**（不是概率），target=0 时 loss=**0.8926**；target=1 时 **1.5926**；target=2 时 **1.6326**；target=3 时 **1.6526**。target 越"错"，loss 越大。

4.3 Gradient descent

```
θ_new = θ_old - η ∇L(θ_old)
```

- `η`：learning rate（step size）。
- **η 太小**：收敛慢，需要很多 epoch。
- **η 太大**：反复越过最小值 → loss 振荡/上升，甚至发散。
- GD 可能停在 **local minimum / saddle point / plateau**。
- 实际中几乎不用 full-batch GD；用 **mini-batch GD**（32–256）或 **SGD**（单样本；库里的 SGD 常指 mini-batch + momentum）。

4.4 Momentum 与 Adam

- **Momentum**：用历史梯度平滑更新，越过小的局部极小/鞍点。
- **Adam**：一阶矩（梯度均值）+ 二阶矩（梯度平方均值）自适应学习率；最常用。
- 其他：Adagrad、Adadelta、RMSprop、Nadam。

4.5 Backpropagation（必背）

对 2 层网络（hidden + output），sigmoid activation，MSE loss：

```
δ^(2) = (a^(2) - y) ⊙ f'(z^(2))
δ^(1) = (W^(2))^T δ^(2) ⊙ f'(z^(1))
∂L/∂W^(2) = δ^(2) (a^(1))^T
∂L/∂W^(1) = δ^(1) x^T
∂L/∂b^(2) = δ^(2)
∂L/∂b^(1) = δ^(1)
```

> 说明：上面按 `L = 1/2 (a - y)^2` 写；若课程用 `L = (a - y)^2`，`δ^(2)` 会多一个因子 2，可被 learning rate 吸收，不影响概念题。

⭐ 直觉：

- `δ` 是"该层对 loss 的敏感度"；
- 从输出往输入传，每层乘上 `W^T` 和 activation derivative；

- sigmoid 的 `f'(z)=a(1-a)`；tanh 的 `f'(z)=1-a^2`；ReLU 的 `f'(z)=1`（z>0）或 0（z<0）。

4.6 Epoch / Batch / Iteration

术语	含义
Epoch	遍历整个训练集一次
Batch / mini-batch	一次参数更新用的样本集合
Iteration	一次参数更新；每个 epoch 有 `ceil(m / B)` 次 iteration（m 样本，B batch size）

4.7 Learning rate scheduling

- **Decay**：随训练降低 learning rate（step、exponential、cosine）。
- **Warmup**：先逐渐增大，再 decay。
- 作用：前期快速学习，后期稳定收敛。

4.8 Vanishing gradient

- 深层网络中，梯度逐层相乘；sigmoid/tanh 的导数 <1，连乘后→0。
- 解决：**ReLU**、**batch normalization**、合适的初始化、residual connection。
- 对应还有 **exploding gradient**（梯度→很大）；可用 gradient clipping。

4.9 Week 10 高频陷阱

陷阱	正确说法
"sigmoid 输入很大时输出变负"	❌ sigmoid ∈ (0,1)；大正输入→1
"learning rate 越大收敛越快"	❌ 太大会振荡/发散
"softmax 输出 one-hot"	❌ softmax 是连续概率；argmax 才是 one-hot
"softmax 每个输出独立"	❌ 分母耦合所有 logits
"cross-entropy 的输入必须是概率"	❌ PyTorch `cross_entropy` 默认输入 **logits**
"ReLU 正半轴饱和"	❌ ReLU 正半轴导数为 1
"dropout 在 inference 也随机丢弃"	❌ 常规 inference 关闭 dropout

---

5. Week 11 — Training（后半）+ CNN 导论

5.1 Generalization：underfitting vs overfitting

	Training loss	Validation loss	原因
Underfitting	高	高	模型太简单 / 训练不够
Just right	低	低	容量合适、训练充分
Overfitting	很低（≈0）	高（先降后升）	容量太大 + 训练太久；记住 noise

⭐ **最清晰的 overfit 信号：training loss 继续下降，validation loss 停止下降并开始上升。**

- 两条曲线都高且平 → underfit；
- 两条都降到低值 → 泛化良好；
- 同步振荡 → 优化不稳定（learning rate 太大），不是 overfit 的典型信号。

5.2 Regularization

L2（weight decay）：

```
J_reg = J_data + λ Σ_k w_k^2
```

- 惩罚大权重；λ 越大，权重越小。
- 有闭式解（线性模型）：`w = (X^T X + λI)^(-1) X^T y`。
- 太小 → 正则不足；太大 → underfit。
- PyTorch：`optimizer = SGD(..., weight_decay=0.01)`。

L1：

```
J_reg = J_data + λ Σ_k |w_k|
```

- 倾向于把不重要的权重压到 **exactly 0** → **sparse / feature selection**。
- 在 NN 中不如 L2 常用；0 点不可导。
- L1 + L2 = **elastic net**。

⭐ **L1 vs L2：**

- L1 → 稀疏（部分权重为 0）；
- L2 → 权重整体变小，但通常不恰好为 0。

5.3 Dropout

- 训练时每个 neuron 以概率 `p` **独立丢弃**（常 20%–50%）。
- 作用：减少参数/复杂度；可看作训练很多子网络并 **ensemble**。
- ⭐ **Inference 时必须关闭**：`model.eval()`；否则同一输入输出随机。
- **经典 dropout**：训练时 `h_train = m * h`；推理时 `h_infer = (1-p) * h`（乘 keep probability，保持期望一致）。

- **Inverted dropout**（现在更常用）：训练时 `h_train = (m / (1-p)) * h`；推理时 `h_infer = h`（不再缩放）。
- ⭐ 题目若说 "inference 乘 keep probability"，默认是**经典 dropout**；若说 "inference 不缩放"，是 inverted dropout。两种都在推理时**关闭随机丢弃**。

5.4 Early stopping

- 从 training data 分出一部分作 **validation set**（如 75/25）。
- 当 validation loss/accuracy 连续 `n` 个 epoch 没有改善时停止；`n` 叫 **patience**。
- 目的：在 validation 开始变差前停下，防止 overfit。
- Validation 不参与 backprop；test 完全不碰。

5.5 Batch normalization

对每个 mini-batch、每个 feature/channel：

```
μ_B = (1/B) Σ_i x_i
σ_B^2 = (1/B) Σ_i (x_i - μ_B)^2
x_hat = (x - μ_B) / sqrt(σ_B^2 + ε)
y = γ x_hat + β
```

- `γ, β` 是可学习的 scale/shift。
- 作用：稳定训练、减少 internal covariate shift、允许更大 learning rate、缓解 vanishing/exploding gradient。
- **Training**：用当前 batch 的 μ/σ；**Inference**：用训练时累计的 running mean/var。
- 常配合 CNN 使用。
- ⭐ BN 不改变激活函数、不强制权重矩阵范数为 1、不"随机打乱 neuron"。

5.6 Hyperparameter tuning

方法	做法	特点
Grid search	每个超参数候选值的笛卡尔积	可靠但组合爆炸
Random search	随机采样组合	更高效，常优先
Bayesian optimization	用代理模型指导采样	研究热点

K-fold CV（小数据常用）：

1. 先留出 test set；
2. training data 分 K 折；
3. 每次 1 折 validation、K-1 折 training；
4. 重复 K 次，平均 validation performance；
5. 选最优 hyperparameters；
6. 最后在 test set 上评估一次。

5.7 Deep vs shallow

- 深层网络通常比浅层好（同样 neuron 数，分层放置更有效）；
- 但超过一定深度后 performance **plateaus**；
- 深层需要 skip connection / BN / 合适初始化才能训练。

5.8 CNN 导论

为什么 MLP 不适合图像：

- 图像平移（translation）后语义不变，但 MLP 的输入向量每个位置都变了；
- MLP **不具备 translation invariance**；同一个 object 在左/右响应不同；
- CNN 用 **local receptive field + parameter sharing** 解决。

Convolution：

```
(f * g)[n] = Σ_k f[k] g[n-k]          (1D 离散)
```

- 2D：filter 在 image 上滑动，element-wise 相乘再求和；
- 数学定义要 **flip kernel**；深度学习框架实际做 **cross-correlation**（不 flip）；因为 filter 是学出来的，两者等价。
- **Local receptive field**：每个 neuron 只连接局部区域；
- **Parameter sharing**：同一个 filter 在整张图共享权重 → 参数少；
- **Feature map depth** = filter 数量；
- **Stride**：滑动步长；**Padding**：边界补零。

Pooling：

- Max pooling / average pooling；
- 降低 spatial size、减少参数、防 overfit。

CNN 架构：

```
input → [conv → ReLU] × N → pool → ... → flatten → FC (MLP) → softmax
```

- convolution 提特征；pooling 降维；FC 分类；
- 参数大多在最后的 FC 层；conv 层参数少（parameter sharing）。

ResNet：

- Identity skip connection：`x_(l+1) = x_l + F(x_l)`；
- 缓解 vanishing gradient；可训练 1000+ 层；
- 常作为 backbone。

5.9 Week 11 高频陷阱

陷阱	正确说法
"L1 让权重整体变小但不为 0"	❌ L1 倾向**稀疏**（部分为 0）；L2 才是整体变小
"dropout 推理时继续随机丢弃"	❌ 推理时关闭；经典实现乘 keep prob，inverted 不乘
"batch norm 在推理时用当前 batch 统计量"	❌ 推理用训练累计的 running mean/var
"BN 消除 vanishing gradient"	❌ 缓解，不是完全消除
"validation loss 上升但 training loss 下降 = underfit"	❌ 这是 **overfit**
"UAT 保证单层网络最好"	❌ UAT 不比较效率/深度
"Conv 数学上不 flip kernel"	❌ 数学定义 flip；深度学习实现不 flip = cross-correlation

---

6. Practice Quiz 2：13 题答案与考点

> 交互版：`6497/quiz2/EE6497_Practice_Quiz_2.html`；逐项解析：`6497/quiz2/Q2_practice_逐项解析.txt`。

题	主题	答案	一句话理由
Q1	Batch normalization	**C**	按 mini-batch 归一化到零均值/单位方差，稳定训练、允许更大 LR
Q2	Dying ReLU	**B**	负半轴梯度为 0，权重收不到更新
Q3	Single-layer perceptron & XOR	**D**	单层只能线性边界；XOR 线性不可分
Q4	L2 regularization	**B**	惩罚平方权重，抑制大权重
Q5	Leaky ReLU	**D**	负半轴给小斜率 α，保留小梯度
Q6	"2-layer" terminology	**A**	1 hidden + 1 output；input 不计
Q7	tanh vs sigmoid	**E**	tanh zero-centered（-1,1），梯度更新更平衡
Q8	Overfitting curves	**D**	training loss 降、validation loss 升
Q9	Universal Approximation	**B**	足够多 hidden neurons 可逼近紧致域上连续函数
Q10	Sigmoid saturation	**C**	饱和区导数→0 → vanishing gradient
Q11	Softmax	**A**	输出非负、和为 1 的概率分布
Q12	Learning rate too large	**E**	反复越过最小值 → loss 振荡/上升
Q13	Dropout at inference	**F**	推理关闭 dropout；经典实现乘 keep prob（inverted 不乘）

6.1 重点题精讲

Q1（Batch norm）：BN 对每个 mini-batch 的激活做 $x_hat = (x-\mu_B)/\sqrt{\sigma_B^2+\varepsilon})$，再 $y=\gamma x_hat+\beta$。主要收益是稳定训练、允许更大 LR。它**不**替代 activation、**不**消除 bias 的必要性、**不**等于 dropout、**不**强制权重 Frobenius norm=1。

Q2（Dying ReLU）：ReLU 在 `z<0` 时导数 0；若某 neuron 对所有相关样本持续 `z<0`，它的权重收不到梯度，输出恒 0。注意"永久"有条件：前层参数变化可能让它重新进入正半轴。

Q4（L2）：`J_reg = J_data + λ Σ w²`；大权重被更强地拉回 0。L1 才是 `Σ|w|`、产生稀疏；L0 才是非零个数；往权重更新里加高斯噪声是另一回事。

Q7（tanh vs sigmoid）：tanh 值域 (-1,1)、zero-centered；sigmoid 值域 (0,1)、输出全正。tanh 仍会饱和，不能"完全消除" vanishing gradient。

Q9（UAT）：单 hidden layer + 足够多 neuron + 合适 activation → 可逼近紧致域上任意连续函数到任意精度。UAT 只保证**存在性**，不保证 GD 找到、不保证泛化、不保证单层更高效、不保证每次训练结果相同。

Q10（Sigmoid saturation）：`σ'(z)=σ(z)(1-σ(z))`；z→±∞ 时 σ→0/1，导数→0；链式法则连乘 → 早层梯度消失。sigmoid 在所有有限 z 上可微，不会"不可微"。

Q11（Softmax）：`p_i=e^(z_i)/Σ_j e^(z_j)`；`p_i>0`、`Σp_i=1`。它**不是** one-hot（argmax 才是）、**不是** 每个 logit 除以 K、**不是** 独立 sigmoid、**不是** 除以 max logit、**不是** tanh。

Q12（LR too large）：`θ_new=θ_old-η∇L`；η 太大会反复越过最小值，loss 振荡/上升/发散。η 太小才是"每步很小、收敛慢"。大 LR 不保证"立即记住训练集"或"逃离所有局部极小"。

Q13（Dropout inference）：常规推理**关闭**随机丢弃，所有 neuron 参与。经典 dropout 推理时乘 keep probability `q=1-p` 以匹配训练期望；inverted dropout 训练时已除以 q，推理不乘。题目若默认经典实现，选"关闭 + 乘 q"。

---

7. A4 Reference Sheet（可直接抄）

> Quiz 2 允许 **一张双面 A4**；下面按"正面/背面"组织，建议考前手抄一遍。

正面：基础 + 前向/反向

1. Neuron / layer

```
z = W x + b
a = f(z)
a^(l) = f( W^(l) a^(l-1) + b^(l) )
```

2. Activations

函数	公式	值域	导数
Sigmoid	`1/(1+e^-z)`	(0,1)	`a(1-a)`
Tanh	`2σ(2z)-1`	(-1,1)	`1-a^2`
ReLU	`max(0,z)`	[0,∞)	`1 (z>0)`, `0 (z<0)`
Leaky ReLU	`z` / `αz`	(-∞,∞)	`1 (z>0)` / `α (z<0)`
Softmax	`e^zi/Σe^zj`	(0,1), Σ=1	—

3. Layers / regions

层数	区域
1 层	half-plane
2 层（1 hidden）	convex region
3 层（2 hidden）	arbitrary region

4. Forward example

```
z1 = W1 x + b1
a1 = sigmoid(z1)
z2 = W2 a1 + b2
a2 = softmax(z2)
```

5. Backprop（2 层，sigmoid + MSE）

```
δ2 = (a2 - y) ⊙ f'(z2)
δ1 = (W2)^T δ2 ⊙ f'(z1)
∂L/∂W2 = δ2 (a1)^T
∂L/∂W1 = δ1 x^T
∂L/∂b2 = δ2
∂L/∂b1 = δ1
```

6. Loss

```
CE = -(1/N) Σ Σ y log p
MSE = (1/N) Σ (y - y_hat)^2
```

背面：训练 + 正则 + CNN

7. GD / optimizers

```
θ_new = θ_old - η ∇L
Momentum: v = βv + ∇L; θ -= ηv
Adam: m_t, v_t 一阶/二阶矩 + bias correction
```

8. Epoch / batch

```
1 epoch = 遍历整个训练集一次
1 iteration = 一次参数更新
iterations per epoch = ceil(m / B)
```

9. Regularization

```
L2: J + λ Σ w^2        (weight decay)
L1: J + λ Σ |w|        (sparse)
Elastic net: L1 + L2
```

10. Dropout

```
训练: mask ~ Bernoulli(1-p); h_train = mask * h
经典推理: h_infer = (1-p) * h
Inverted: 训练 h_train = mask*h/(1-p); 推理 h_infer = h
推理必须 model.eval()
```

11. Batch norm

```
μ_B = mean(x); σ_B^2 = var(x)
x_hat = (x - μ_B) / sqrt(σ_B^2 + ε)
y = γ x_hat + β
训练: batch stats; 推理: running stats
```

12. Overfitting

```
train loss ↓, val loss ↓ then ↑  → overfit
both high/flat → underfit
both low → good
```

13. CNN

```
conv: y[i,j] = Σ_m Σ_n x[i+m, j+n] * w[m,n]   (cross-correlation in DL)
pool: max / average; stride
CNN: conv → ReLU → pool → ... → FC → softmax
ResNet: x_(l+1) = x_l + F(x_l)
```

14. UAT

```
1 hidden layer + enough neurons + suitable activation
→ approximate any continuous function on compact domain to arbitrary accuracy
（存在性，不保证优化/泛化）
```

---

8. 考前 1 小时：20 条高频判断

#	陈述	对/错
1	2-layer network = 2 hidden layers	❌ 1 hidden + output
2	Input layer 计入层数	❌ 不计
3	Sigmoid 输出可以为负	❌ (0,1)
4	Tanh 是 zero-centered	✅
5	Tanh 不会饱和	❌ 仍饱和
6	ReLU 正半轴饱和	❌ 导数为 1
7	Leaky ReLU 负半轴梯度为 α	✅
8	Dying ReLU 根因是负半轴梯度 0	✅
9	单层多 neuron 可解 XOR	❌ 需要 hidden layer + 非线性
10	2 层网络可表示任意区域	❌ 只能凸区域；3 层才任意
11	UAT 保证 GD 找到全局最优	❌ 只保证存在性
12	Softmax 输出和为 1	✅
13	Softmax 等于 one-hot	❌ argmax 才是
14	Cross-entropy 输入默认是 logits（PyTorch）	✅
15	Learning rate 太大 → loss 振荡/发散	✅
16	L1 倾向稀疏，L2 整体缩小	✅
17	Dropout 推理时继续随机丢弃	❌ 关闭；经典乘 keep prob
18	BN 推理用当前 batch 统计量	❌ 用 running stats
19	Overfit：train loss ↓, val loss ↑	✅
20	CNN 中 conv 实际是 cross-correlation	✅（不 flip kernel）

9. 3 天复习计划

Day 1：W8 + W9（基础与表达力）

- ☐ 背 activation 表（sigmoid/tanh/ReLU/Leaky ReLU/softmax）。
- ☐ 手画 XOR 的 3-neuron 解；手算 4+2+1 的 AND/OR 逻辑。
- ☐ 背 1/2/3 层决策区域；UAT 的"保证/不保证"。
- ☐ 做 Practice Quiz Q3、Q6、Q9。

Day 2：W10 + W11（训练与正则）

- ☐ 手算 forward pass（2 层，sigmoid/softmax）。

- ☐ 背 backprop 的 δ 公式；手算一个 2 层网络的梯度。
- ☐ 背 L1/L2、dropout（train vs inference）、BN、early stopping。
- ☐ 做 Practice Quiz Q1、Q2、Q4、Q5、Q7、Q8、Q10、Q12、Q13。

Day 3：模拟 + A4 sheet

- ☐ 计时做 Practice Quiz 13 题（不查资料）。
- ☐ 对错题回看 §2–§5 对应小节。
- ☐ 手抄 §7 的 A4 sheet（正面/背面）。
- ☐ 最后过一遍 §8 的 20 条判断。

---

10. 一句话总结

> **W8–W9 解决"网络能表示什么"（activation / XOR / 层数 / UAT）；W10 解决"怎么训练"（forward / loss / GD / backprop / optimizer）；W11 解决"怎么不过拟合、怎么调、CNN 怎么来"（正则 / dropout / BN / 调参 / convolution / pooling）。Quiz 2 的 MCQ 主要考这些概念的边界和几个手算公式——把 Practice Quiz 13 题 + §7 A4 sheet 吃透，基本就稳了。**